PROJECT

Daily System Resource Usage Report Automation

OPERATING SYSTEM

BCSE303L

SLOT: F1

TEAM MEMBERS

Tarunika Anand	23BCE1203
Mahima Arun Kavitha	23BCE5117
Aishwarya Ramesh	23BCE1083

INDEX

ABSTRACT

This project presents a comprehensive solution for monitoring system performance, designed to track critical resources such as CPU, memory, and disk usage. The system utilizes a shell script (system_monitor.sh) that runs at scheduled intervals, collecting key data about the system's health, including uptime, operating system details, CPU usage, memory consumption, and disk I/O statistics. The script also includes threshold-based alerts: if any resource usage exceeds predefined limits (e.g., CPU > 80%, Memory > 80%, or Disk usage > 80%), an email is automatically sent to notify the user of the issue.

To enhance usability, the project features a Flask-based REST API that serves system data in JSON format to a real-time web dashboard. The dashboard, built with HTML, CSS, and JavaScript, provides an interactive interface for users to visualize up-to-date system information and resource usage. The data displayed on the dashboard is pulled directly from the JSON file, which is updated each time the shell script runs.

The solution automates system monitoring, ensuring that users are promptly informed of any performance issues via email alerts. The combination of shell scripting, REST API, email notifications, and a dynamic web interface makes this project a robust and user-friendly system management tool, capable of providing real-time insights into system health while automating routine monitoring tasks.

INTRODUCTION

System monitoring is a critical task for maintaining the health and stability of both personal and enterprise-level computing environments. As systems become more complex, with multiple processes running simultaneously and vast amounts of data being processed, ensuring that resources such as CPU, memory, and disk space are properly utilized becomes increasingly important. The inability to efficiently monitor and manage these resources can lead to degraded performance, system crashes, or even data loss. Proactively addressing these issues helps prevent downtime, enhances productivity, and supports system optimization.

This project aims to provide an efficient, automated system monitoring solution using a combination of shell scripting, web technologies, and real-time data visualization. The core of this project is a shell script that monitors critical system resources, such as CPU, memory, and disk usage, while also sending alerts when usage exceeds defined thresholds. The script collects detailed system information, stores it in a log file, and sends out email notifications to the system administrator, keeping them informed of any potential resource bottlenecks or failures.

To enhance the user experience and accessibility, a Flask-based web application is incorporated into the project. The application provides a dynamic web dashboard that fetches and displays real-time system information, including uptime, resource usage, and alerts. This dashboard auto-refreshes every minute, providing administrators with up-to-date insights into the health of their systems. The integration of a REST API enables remote access to system data, allowing users to fetch the most current system metrics via an API call, thus making this tool flexible and easily extensible for future integrations.

The use of shell scripting in this project provides several advantages. It allows for lightweight, fast execution of system-level commands, making it ideal for continuous monitoring. Shell scripts also integrate seamlessly with system utilities, enabling easy data retrieval and logging. Furthermore, they can be easily scheduled to run at regular intervals using cron jobs, automating the entire monitoring process. This automation reduces the need for manual checks and allows system administrators to focus on more critical tasks while ensuring that system performance remains optimal.

The inclusion of automated email alerts in this project further enhances its functionality. By automatically notifying administrators when resource usage exceeds predefined thresholds, the system helps detect potential problems before they escalate into serious issues. This proactive approach to system management is essential for minimizing downtime and maximizing the overall efficiency of IT operations.

What makes this project innovative is the integration of modern web technologies with traditional system monitoring tools. While system monitoring itself is not a new concept, the combination of a real-time dashboard, automated alerts, and a REST API makes this system more interactive, accessible, and proactive. The web dashboard not only makes monitoring easier but also allows for real-time data access, making it more user-friendly for both technical and non-technical users. The system's flexibility, scalability, and ease of use make it a valuable tool for administrators of all levels, ensuring that they are always equipped with the latest system information.

In conclusion, this project offers a comprehensive, automated, and user-friendly solution for monitoring system health. By combining the power of shell scripting, web technologies, and automation, it provides a flexible and scalable system that can be easily integrated into various IT environments. The system's ability to monitor resources, send email alerts, and display real-time data through a web dashboard makes it an invaluable tool for anyone responsible for maintaining the health of a computer system

METHODOLOGY

The methodology for developing this system monitoring tool follows a structured approach, integrating multiple technologies to ensure that system performance is continually tracked, alerts are sent automatically when necessary, and the monitoring results are made accessible through a dynamic web interface. The key steps of the methodology are as follows:

- **System Information Collection (Shell Script):** The first step is to gather crucial system information, such as CPU usage, memory consumption, disk usage, and uptime. This is done through a shell script (system_monitor.sh), which utilizes built-in Linux system commands to fetch real-time metrics.

  These commands include:

  - top for CPU usage
  - free for memory usage
  - iostat for disk usage

  The shell script also collects system details like the hostname, IP address, operating system information, and kernel version using commands like hostname, lsb_release, and uname. This data is then formatted and written to a log file (system_monitor.log), which serves as a historical record of system metrics.

- **Threshold-Based Alerts:** The shell script includes predefined thresholds for CPU, memory, and disk usage (set to 80% in this project). If any of these thresholds are exceeded, an alert is generated and appended to the log file. This enables the system administrator to quickly identify when a resource is under heavy usage, which may indicate potential performance issues. If no issues are detected, a message confirming that the system is running smoothly is appended to the log.

- **Automated Email Alerts:** When a threshold is crossed, the shell script uses the mail command to send an email notification to the administrator, providing them with detailed information about the system's resource usage. The script dynamically generates the subject and body of the email, which includes the most recent system data and the resource usage details that triggered the alert. The email is sent to the designated recipient, keeping the system administrator informed in real-time.

- **JSON Data Formatting:** In addition to logging system information in the text-based log file, the script also creates a JSON file (system_info.json) containing the same system details in a structured format. This JSON file serves as the data source for the web dashboard and API. The file is updated every time the script runs, ensuring that the most recent system data is always available for the web interface.

- **Web Dashboard (Flask Application):** A Flask-based Python web application is developed to serve as the user interface for displaying system information. The web application provides a real-time dashboard, which fetches data from the system_info.json file through an API endpoint. The front end of the dashboard is built using HTML and JavaScript, where the fetchData() function periodically retrieves updated system metrics from the Flask API every minute. This ensures that the data presented on the dashboard is always current.

- **REST API:** The Flask application includes a REST API endpoint (/api/system_info), which serves the system information stored in the system_info.json file. This API is designed to be lightweight and provides the flexibility to fetch the system data programmatically from any external system or application. By using the jsonify() function from Flask, the API returns the system data in JSON format, making it easy for developers to integrate the monitoring system with other applications or tools if needed.

- **Auto-Refresh Dashboard:** The web dashboard is designed to automatically refresh every minute. This is achieved using JavaScript's setInterval() function, which triggers the fetchData() function at regular intervals. This ensures that the user always sees the latest system information without needing to manually refresh the page.

- **Cron Job Automation:** To ensure that the shell script runs at regular intervals, a cron job is configured on the system to execute the system_monitor.sh script automatically at a scheduled time, such as every minute or every day at a specific time. This automation ensures continuous system monitoring with no manual intervention required.

- **CSS Styling for Dashboard:** The dashboard's appearance is enhanced using CSS. The styles.css file defines the layout, color scheme, and visual style of the dashboard, making it easy to read and visually appealing. The header, containers, and individual sections of the dashboard are styled to create a user-friendly interface.

Flow of the Methodology:

- The system_monitor.sh script runs at predefined intervals (via cron job), collecting system information.
- The data is saved to a log file and formatted into a JSON file.
- If any thresholds are exceeded, an email alert is sent to the administrator.
- The Flask web application periodically fetches the JSON data via the /api/system_info endpoint.
- The dashboard updates every minute to display the latest system data in an easy-to-understand format.
- The entire system is automated and requires minimal manual input once set up.

Innovative Aspects:

This system combines traditional shell scripting with modern web technologies to provide an efficient, automated, and user-friendly solution for system monitoring. The integration of a dynamic web dashboard and REST API ensures that system administrators can monitor their systems in real time, while automated alerts reduce the need for constant manual checking. This innovative approach enhances both the usability and functionality of system monitoring tools.

SOURCE CODE

1. SHELL SCRIPT: system_monitor.sh

```
#!/bin/bash

log_file="./system_monitor.log"
echo "" > $log_file

cpu_threshold=80.0
mem_threshold=80.0
disk_user_threshold=80.0

center_heading() {
    local heading="$1"
    local width=100
    local padding=$(($width - ${#heading} - 2))
    local hyphens=$(printf '%*s' "$((padding / 2))" | tr ' ' '-')
    printf "%s %s %s\n" "$hyphens" "$heading" "$hyphens"
}

hostname=$(hostname)
ip_address=$(hostname -I | awk '{print $1}')
uptime=$(uptime -p)
```

```
os_info=$(lsb_release -d | awk '{print $2, $3, $4}')
kernel_version=$(uname -r)
architecture=$(uname -m)
processor=$(lscpu | grep "Model name" | cut -d: -f2 | xargs)
cpu_cores=$(nproc)
total_ram=$(free -h | awk '/^Mem:/ {print $2}')

center_heading "System Information" >> $log_file
echo "Hostname        : $hostname" >> $log_file
echo "IP Address      : $ip_address" >> $log_file
echo "Uptime          : $uptime" >> $log_file
echo "Operating System: $os_info" >> $log_file
echo "Kernel Version  : $kernel_version" >> $log_file
echo "Architecture    : $architecture" >> $log_file
echo "Processor       : $processor" >> $log_file
echo "CPU Cores       : $cpu_cores" >> $log_file
echo "Total RAM       : $total_ram" >> $log_file
echo "" >> $log_file

cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')
mem_usage=$(free | awk '/^Mem:/ {print $3/$2 * 100.0}')
disk_usage=$(iostat -c | awk 'NR==4 {print $1, $2, $3, $6}')

alerts=""
```

```
if (( $(echo "$cpu_usage > $cpu_threshold" | bc -l) )); then
    alerts+="ALERT: CPU usage is high ($cpu_usage%)\n"
fi

if (( $(echo "$mem_usage > $mem_threshold" | bc -l) )); then
    alerts+="ALERT: Memory usage is high ($mem_usage%)\n"
fi

disk_user=$(echo $disk_usage | awk '{print $1}')
disk_system=$(echo $disk_usage | awk '{print $2}')
disk_iowait=$(echo $disk_usage | awk '{print $3}')
disk_idle=$(echo $disk_usage | awk '{print $4}')

if (( $(echo "$disk_user > $disk_user_threshold" | bc -l) )); then
    alerts+="ALERT: Disk user usage is high ($disk_user%)\n"
fi

center_heading "Resource Usage" >> $log_file
echo "CPU Usage       : $cpu_usage%" >> $log_file
echo "Memory Usage    : $mem_usage%" >> $log_file
center_heading "Disk Usage" >> $log_file
echo "  User   : $disk_user%" >> $log_file
echo "  System : $disk_system%" >> $log_file
echo "  Iowait : $disk_iowait%" >> $log_file
echo "  Idle   : $disk_idle%" >> $log_file
```

```
echo "" >> $log_file

if [ -z "$alerts" ]; then
   echo "System status: All systems are operating within normal parameters." >> $log_file
else
   echo -e "$alerts" >> $log_file
fi

email_subject="System Monitoring Report for $hostname"
email_recipient="tarunikaa2005@gmail.com"
email_sender="linuxsender24@gmail.com"

mail -s "$email_subject" -r "$email_sender" "$email_recipient" < $log_file

system_info=$(cat <<EOF
{
   "hostname": "$hostname",
   "ip_address": "$ip_address",
   "uptime": "$uptime",
   "os_info": "$os_info",
   "kernel_version": "$kernel_version",
   "architecture": "$architecture",
   "processor": "$processor",
   "cpu_cores": "$cpu_cores",
   "total_ram": "$total_ram",
```

```
    "cpu_usage": "$cpu_usage%",
    "memory_usage": "$mem_usage%",
    "disk_usage": {
        "user": "$disk_user%",
        "system": "$disk_system%",
        "iowait": "$disk_iowait%",
        "idle": "$disk_idle%"
    }
}
EOF
)

echo "$system_info" > /mnt/c/project/system_info.json
```

2.PYTHON SCRIPT: app.py

```
from flask import Flask, render_template, jsonify
import json
import os
app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/api/system_info')
def system_info():
    try:
        json_file_path = '/mnt/c/project/system_info.json'
        if os.path.exists(json_file_path):
            with open(json_file_path, 'r') as file:
                system_data = json.load(file)
            return jsonify(system_data)
        else:
            return jsonify({'error': 'system_info.json file not found'}), 404
    except Exception as e:
        return jsonify({'error': str(e)}), 500
if __name__ == '__main__':
    app.run(debug=True)
```

3.JSON SCRIPT: system_info.json


```
{
   "hostname": "DESKTOP-CCV0DMO",
   "ip_address": "172.31.224.1",
   "uptime": "up 21 hours, 38 minutes",
   "os_info": "Ubuntu 22.04.5 LTS",
   "kernel_version": "4.4.0-19041-Microsoft",
   "architecture": "x86_64",
   "processor": "Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz",
   "cpu_cores": "4",
   "total_ram": "7.9Gi",
   "cpu_usage": "8.1%",
   "memory_usage": "79.5161%",
   "disk_usage": {
      "user": "16.35%",
      "system": "0.00%",
      "iowait": "13.48%",
      "idle": "70.17%"
   }
}
```

4. HTML FILE: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>System Monitoring Dashboard</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    <header>
        <i class="fa fa-cogs"></i>
        <span>System Monitoring Dashboard</span>
    </header>

    <div class="container">
        <h2>System Information</h2>
        <div class="info-box" id="system-info"></div>

        <h2>Resource Usage</h2>
        <div class="info-box" id="resource-usage"></div>

        <h2>Disk Usage</h2>
        <div class="info-box" id="disk-usage"></div>
```

```
        <h2>Status</h2>
        <div class="status-box">
            <p id="status-message">Loading...</p>
        </div>

<script>
function fetchData() {
    fetch('/api/system_info')
        .then(response => response.json())
        .then(data => {
            if (data.error) {
                document.getElementById('status-message').innerText = 'Error loading
system information: ' + data.error;
            } else {
                document.getElementById('system-info').innerHTML = `
                    <p><strong>Hostname:</strong> ${data.hostname}</p>
                    <p><strong>IP Address:</strong> ${data.ip_address}</p>
                    <p><strong>Uptime:</strong> ${data.uptime}</p>
                    <p><strong>Operating System:</strong> ${data.os_info}</p>
                    <p><strong>Kernel Version:</strong> ${data.kernel_version}</p>
                    <p><strong>Processor:</strong> ${data.processor}</p>
                    <p><strong>CPU Cores:</strong> ${data.cpu_cores}</p>
                    <p><strong>Total RAM:</strong> ${data.total_ram}</p>
                `;
```

```
            document.getElementById('resource-usage').innerHTML = `
                <p><strong>CPU Usage:</strong> ${data.cpu_usage}</p>
                <p><strong>Memory Usage:</strong> ${data.memory_usage}</p> `;

            document.getElementById('disk-usage').innerHTML = `
                <p><strong>User:</strong> ${data.disk_usage.user}</p>
                <p><strong>System:</strong> ${data.disk_usage.system}</p>
                <p><strong>Iowait:</strong> ${data.disk_usage.iowait}</p>
                <p><strong>Idle:</strong> ${data.disk_usage.idle}</p>
            `;
            document.getElementById('status-message').innerText = 'System information loaded successfully.';
        }
    })
    .catch(error => {
        document.getElementById('status-message').innerText = 'Error fetching data: ' + error;
    });
}
setInterval(fetchData, 60000);
fetchData();
</script>
  </div>
</body>
</html>
```

5. CSS FILE: styles.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f9;
    color: #333;
}

header {
    background: linear-gradient(135deg, #3498db, #9b59b6);
    padding: 20px;
    border-radius: 10px;
    text-align: center;
    color: white;
    font-size: 2.5em;
    font-weight: bold;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
}
```

```
header i {
    margin-right: 10px;
    font-size: 1.5em;
}

header span {
    font-size: 1.2em;
    color: #f0f0f0;
}
.container {
    max-width: 1000px;
    margin: 30px auto;
    padding: 20px;
    background-color: white;
    border-radius: 10px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

h2 {
    color: #3498db;
    font-size: 1.8em;
    margin-bottom: 10px;
}
```

```
.info-box, .status-box {
    background-color: #ecf0f1;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
}

.info-box p, .status-box p {
    font-size: 1.1em;
    margin: 5px 0;
}

.status-box {
    background-color: #dff0d8;
}

.status-box p {
    font-weight: bold;
    text-align: center;
    color: #3c763d;
}

.status-box.alert {
    background-color: #f2dede;
    color: #a94442;
```

```
}

strong {
    color: #2980b9;
}
```

IMPLEMENTATION

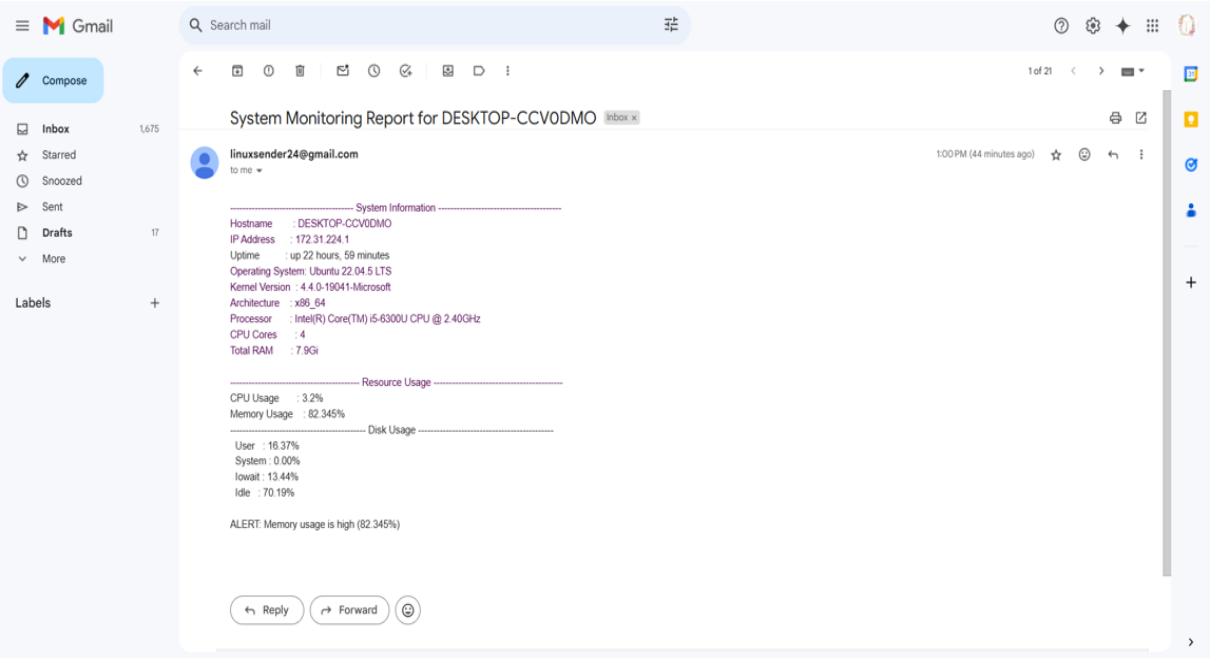
1. CRONTAB SETTING AT 1.00 PM FOR AUTOMATIC SCRIPT EXECUTION:

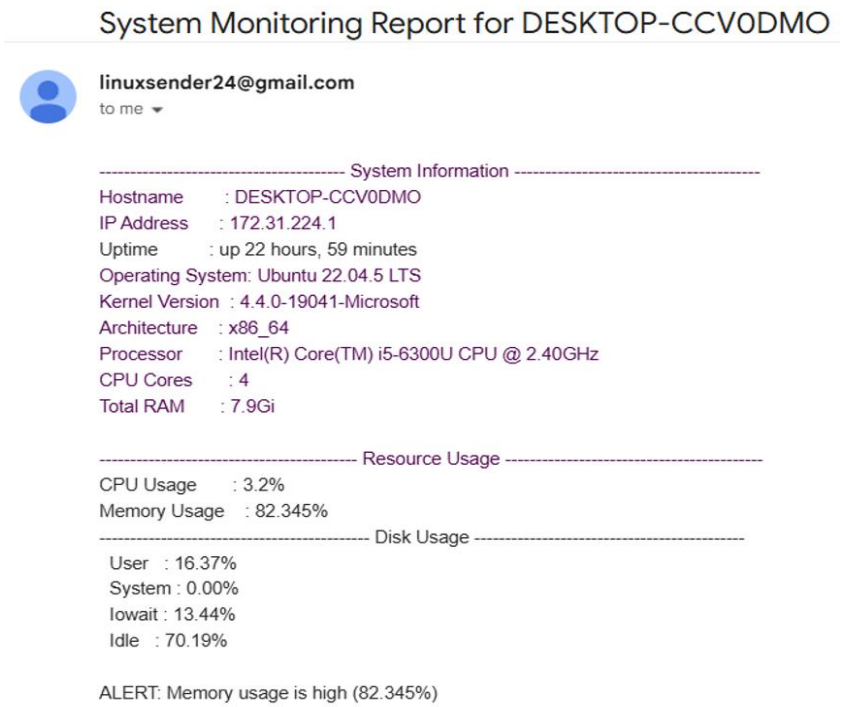
```
tarunika05@DESKTOP-CCV0DMO:/mnt/c/project/static$ crontab -l

0 13 * * * /mnt/c/project/system_monitor.sh
```

2. UPDATED LOG FILE:

```
tarunika05@DESKTOP-CCV0DMO:/mnt/c/project$ cat system_monitor.log

---------------------------------------- System Information ----------------------------------
Hostname        : DESKTOP-CCV0DMO
IP Address      : 172.31.224.1
Uptime          : up 22 hours, 59 minutes
Operating System: Ubuntu 22.04.5 LTS
Kernel Version  : 4.4.0-19041-Microsoft
Architecture    : x86_64
Processor       : Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz
CPU Cores       : 4
Total RAM       : 7.9Gi

---------------------------------------- Resource Usage ----------------------------------
CPU Usage       : 3.2%
Memory Usage    : 82.345%
---------------------------------------- Disk Usage ----------------------------------
  User   : 16.37%
  System : 0.00%
  Iowait : 13.44%
  Idle   : 70.19%

ALERT: Memory usage is high (82.345%)
```

3. EMAIL SENT AT 1.00 PM





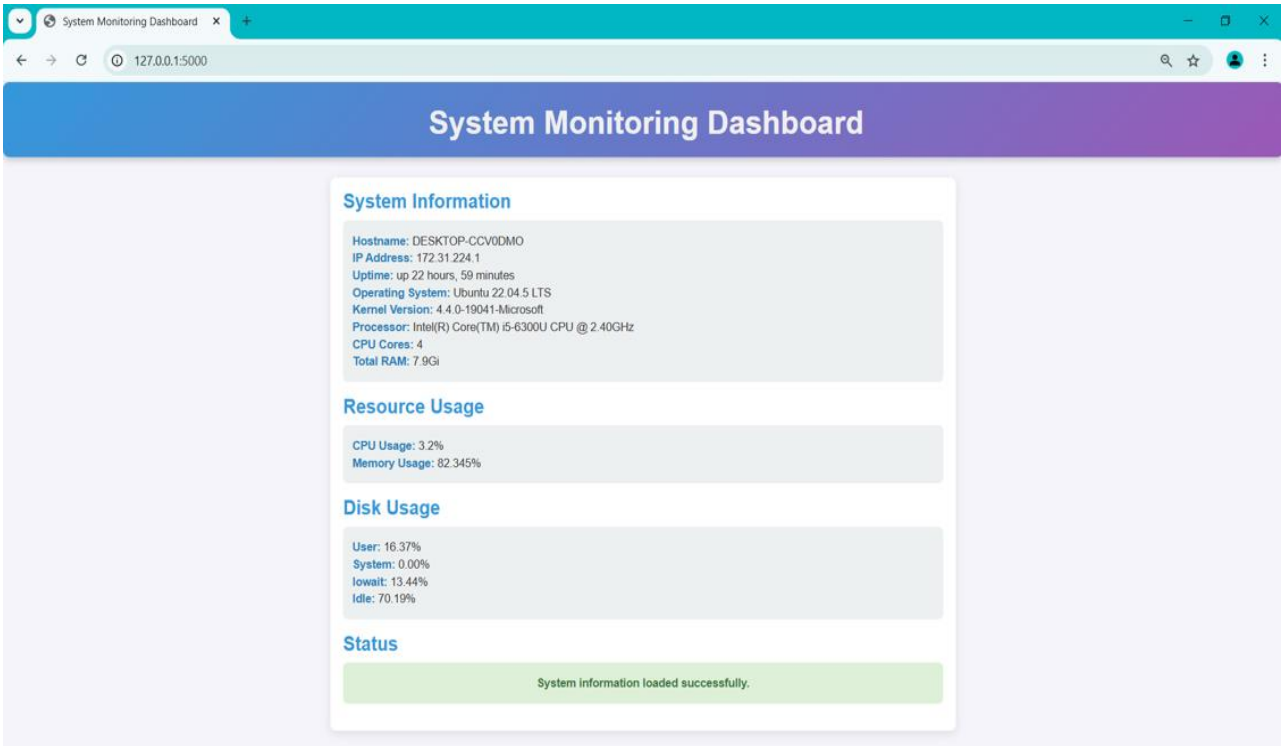
4. UPDATED JSON FILE:

```
tarunika05@DESKTOP-CCV0DMO:/mnt/c/project$ cat system_info.json
{
    "hostname": "DESKTOP-CCV0DMO",
    "ip_address": "172.31.224.1",
    "uptime": "up 22 hours, 59 minutes",
    "os_info": "Ubuntu 22.04.5 LTS",
    "kernel_version": "4.4.0-19041-Microsoft",
    "architecture": "x86_64",
    "processor": "Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz",
    "cpu_cores": "4",
    "total_ram": "7.9Gi",
    "cpu_usage": "3.2%",
    "memory_usage": "82.345%",
    "disk_usage": {
        "user": "16.37%",
        "system": "0.00%",
        "iowait": "13.44%",
        "idle": "70.19%"
    }
}
```

5 . FLASK SERVER RUNNING:

```
^Ctarunika05@DESKTOP-CCV0DMO:/mnt/c/project$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 425-014-466
127.0.0.1 - - [10/Nov/2024 11:35:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Nov/2024 11:35:13] "GET /static/styles.css HTTP/1.1" 304 -
127.0.0.1 - - [10/Nov/2024 11:35:13] "GET /api/system_info HTTP/1.1" 200 -
```

6 . WEB DASHBOARD:



System Monitoring Dashboard

System Information

Hostname: DESKTOP-CCV0DMO
IP Address: 172.31.224.1
Uptime: up 22 hours, 59 minutes
Operating System: Ubuntu 22.04.5 LTS
Kernel Version: 4.4.0-19041-Microsoft
Processor: Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz
CPU Cores: 4
Total RAM: 7.9Gi

Resource Usage

CPU Usage: 3.2%
Memory Usage: 82.345%

Disk Usage

User: 16.37%
System: 0.00%
Iowait: 13.44%
Idle: 70.19%

Status

System information loaded successfully.

REFERENCES

Shell Scripting

1. Shell Script - Basic Guide
2. Bash Guide for Beginners - Bash Guide for Beginners
3. Linux Command Line Basics - Linux Command Line Basics
4. GNU Bash Manual - GNU Bash Manual
5. Linux man pages - Accessed through the terminal: man <command>
6. Setting up Cron jobs on Ubuntu - CronHowto - Ubuntu Wiki
7. Stack Overflow - Cron Jobs Guide on Stack Overflow
8. Linux Mail Command - Linux Mail Command
9. Sending emails using SMTP on Linux - Mailgun

Flask

1. Building RESTful APIs with Flask - RESTful APIs with Flask
2. Flask Documentation - Flask Documentation

Frontend Development

1. HTML, CSS, and JavaScript tutorials: W3Schools
2. JavaScript fetch() API: MDN Web Docs - fetch() API